EXPERIMENT 11

SOFTWARE METRICS AND COMPLEXITY ANALYSIS

Project Name: **Food Delivery Management System**

1. Aim

To study software metrics and perform complexity analysis of the Food Delivery Management System to evaluate its size, complexity, quality, and maintainability.

2. Objectives

• To understand the concept of software metrics.

• To measure the size and complexity of software.

• To calculate Cyclomatic Complexity.

• To understand code quality and maintainability.

• To identify methods for improving software quality.

3. Theory

Software Metrics are quantitative measures used to evaluate different characteristics of software, such as size, complexity, performance, quality, and maintainability. Software metrics help developers understand how complex a system is and how easily it can be tested, maintained, and improved.

Types of Software Metrics

1. Lines of Code (LOC)

Measures the total number of lines in a program. Variants include physical LOC (all lines), logical LOC (executable statements), comment lines and blank lines. It is a simple size measure used for effort and cost estimation, but it depends on coding style and language.

2. Cyclomatic Complexity (CC)

Proposed by Thomas McCabe, it measures the number of linearly independent paths through a program. It indicates the minimum number of test cases required for full branch coverage.

Formulas: **V(G) = E - N + 2P** (E = edges, N = nodes, P = connected components) or **V(G) = Number of decision points + 1**.

CC Value	Risk / Interpretation
1 - 10	Simple, low risk, easy to test
11 - 20	Moderate complexity, moderate risk
21 - 50	High complexity, high risk
> 50	Untestable, very high risk

3. Halstead Metrics

Based on operators and operands in the code. Let n1 = distinct operators, n2 = distinct operands, N1 = total operators, N2 = total operands.

• Vocabulary n = n1 + n2; Length N = N1 + N2

• Volume V = N x log2(n)

• Difficulty D = (n1 / 2) x (N2 / n2)

• Effort E = D x V

4. Maintainability Index (MI)

A composite value: **MI = 171 - 5.2 ln(V) - 0.23 V(G) - 16.2 ln(LOC)**. Higher values mean easier maintenance (above 85 good, 65-85 moderate, below 65 difficult).

5. Coupling and Cohesion

Coupling measures dependency between modules (lower is better). Cohesion measures how closely the elements of a module are related (higher is better).

6. Function Point (FP) Analysis

Measures functionality delivered to the user based on external inputs, outputs, inquiries, internal logical files and external interface files, independent of programming language.

4. Project Overview

The Food Delivery Management System allows customers to browse restaurants, place orders, make payments and track deliveries, while restaurants manage menus and delivery partners are assigned to orders. For analysis, the system is divided into five modules: User/Authentication, Restaurant/Menu, Order, Payment and Delivery.

5. Measurement and Analysis

5.1 Lines of Code (Module-wise)

Module	LOC	Methods	Total CC	Avg CC / Method
User / Authentication	150	6	14	2.3
Restaurant / Menu	200	8	17	2.1
Order	310	12	41	3.4
Payment	180	7	19	2.7
Delivery	240	9	28	3.1
Total	**1080**	**42**	**119**	**2.8**

Note: values are illustrative for the sample project; replace them with figures from your own code using a tool such as Radon, SonarQube or Visual Studio Code Metrics.

5.2 Cyclomatic Complexity Example: calculate_bill()

```
def calculate_bill(items, distance, promo):
    total = 0                          # node 1
    for item in items:                 # node 2 (decision)
        total += item.price * item.qty # node 3
    if distance > 5:                   # node 4 (decision)
```

```
        total += 50                    # node 5
    if promo:                          # node 6 (decision)
        total -= promo.discount        # node 7
    return total                       # node 8
```

Method 1 (decision points): for-loop (1) + if distance (1) + if promo (1) = 3 decisions, so V(G) = 3 + 1 = **4**.

Method 2 (graph): Nodes N = 8; Edges E = 10 (1-2, 2-3, 3-2, 2-4, 4-5, 4-6, 5-6, 6-7, 6-8, 7-8). V(G) = E - N + 2 = 10 - 8 + 2 = **4**.

Both methods agree. Four independent paths exist, so at least 4 test cases are needed, for example: (a) empty items, distance 3, no promo; (b) one item, distance 3, no promo; (c) one item, distance 8, no promo; (d) one item, distance 8, with promo.

5.3 Cyclomatic Complexity Example: place_order()

```
def place_order(order):
    if not order.items:                          # D1
        return "Empty cart"
    for item in order.items:                     # D2
        if item.stock < item.qty:                # D3
            return "Out of stock"
    if order.coupon and order.coupon.valid:      # D4, D5 (and)
        apply_discount(order)
    if order.payment == "CARD" or order.payment == "UPI":  # D6, D7 (or)
        process_online_payment(order)
    else:
        mark_cash_on_delivery(order)
    return "Order placed"
```

Decision points = 7 (if, for, if, if, and, if, or). V(G) = 7 + 1 = **8**, which is within the acceptable range (1-10) but is the most complex method in the Order module and a candidate for refactoring.

5.4 Method-wise Complexity Summary

Method	Module	CC	Risk
login()	User / Auth	3	Low
track_order()	Delivery	2	Low
calculate_bill()	Order	4	Low
assign_driver()	Delivery	6	Low
place_order()	Order	8	Low (upper range)

5.5 Maintainability Index (sample for place_order module)

Assume Halstead Volume V = 1500, average CC = 8 and LOC = 40:

MI = 171 - 5.2 ln(1500) - 0.23(8) - 16.2 ln(40)
MI = 171 - 38.03 - 1.84 - 59.76 = **71.4**

An MI of about 71 lies in the moderate maintainability band (65-85): acceptable, but improvement is advisable.

6. Observations

• The system has about 1080 LOC across 42 methods, so it is a small-to-medium size application.

• Average cyclomatic complexity is 2.8 per method, well within the safe limit of 10.

• The Order and Delivery modules have the highest complexity because they contain most of the business rules.

• place_order() is the most complex method (CC = 8) and should be monitored.

• Maintainability is moderate and can be improved through refactoring and better documentation.

7. Methods to Improve Software Quality

• **Refactoring:** split long methods such as place_order() into validate_cart(), apply_coupon() and process_payment().

• **Reduce nesting:** use guard clauses and early returns instead of deep if-else structures.

• **Replace conditionals:** use strategy or polymorphism for payment types (Card, UPI, COD).

• **Low coupling, high cohesion:** keep each module responsible for one function and communicate through interfaces.

• **Unit testing:** write at least V(G) test cases per method and track code coverage.

• **Code reviews and static analysis:** use tools such as SonarQube, Radon or PMD regularly.

• **Documentation:** maintain comments and design documents to raise the Maintainability Index.

• **Set thresholds:** enforce CC limit (for example 10) in the CI pipeline.

8. Conclusion

Software metrics were studied and applied to the Food Delivery Management System. Size was measured using LOC, complexity using Cyclomatic Complexity (by both the decision-point and graph methods), and maintainability using the Maintainability Index. The system shows low to moderate complexity, with the Order module needing the most attention. Applying refactoring, testing and continuous metric monitoring will improve the quality, testability and maintainability of the system.